关于 AHU 的 PDD 全流程记录

1. 灵感来源

最近看到网上一直很火的 Agent，我一直想试一下，但是自己也不是一名计算机专业的学生，万事开头难，都不知道从哪里下手，所以我将自己的疑问问向了 AI，也就是我现在在使用的 gemini，它给出了我可以转行到互联网，并且可以结合自己的专业知识，做一个面前市面上主流的只是关于暖通方面的 Agent 智能体。现在我从一个小白开始干这件事情，也就是一边开始学基础的 python 语法，一边我找了关于机器学习算法的课程在听，同时我要学习整个 Agent 的搭建流程。于是关于这份对于暖通数据的清洗和分析就开始了。

2. 数据的准备阶段

首先就是我的数据来源，这是一个大问题，很多人会在开始的时候很迷茫，真实可靠的数据怎么去获取，如果获取不了的话，我怎么能够真正去得到真实的数据反馈，于是我就开始了我的第一段 Github 之旅，这真是我第一次使用它，我也强烈建议大家去使用它，它是一个比百度更加权威的网站，里面几乎可以找到你当前想要的全部知识方面的内容，就像一个开放的网络知识库，不论是数据还是你想要的文件或者是代码的源码里面一般都会有的。我去 Github 找了我的这一份初始数据的 excel，数据的来源是一个韩国的空调公司提供的。数据的真伪需要自己去辨别，当然目前为止，我暂时还管不了那么多。于是我就得到了自己的数据。下一部就要准备开始处理数据了。

3. 用到的软件

主要用到的软件是 cursor，还有 vscode。这两款软件是互通的，是我自己目前使用最顺手的编程软件，当然 kiro 也很好用。哈哈哈哈。这三款软件中都可以调用现在比较好的一些大模型的 API 来帮你进行编程的使用。你可以通过自然语言也就是说人话让他们帮你朝着你想要的方向去改动你的代码，一直到最后实现。当然，每款软件是有免费额度的，用的太猛的话是需要充钱的，不过越充钱越猛。

4. 整个数据清洗的全部流程

万事俱备我就开始着手清洗我的数据了。值得一提的是，一定要用 AI 辅助你在所有的工作开始之前**定好自己的工作流**，这个是最重要的。你

要尽可能的明白你需要干哪些步骤，每一步的目的是什么，需要用到什么，最后需要达到什么样的目的，这应该是整个过程中最关键的，可以让你少走很多弯路。（希望加粗可以提醒到自己，顺便放一张自己的数据清洗的流程图）。

- 第三步：pandas自动化数据清洗七步走
  - 1. 数据质量初步诊断（Data Profiling）
    - 在动刀清洗前，先给数据做个“体检”。
    - 操作：统计每一列的缺失率、最大值、最小值、平均值。
    - 暖通关注点：如果某个传感器的缺失率超过 30%，这个传感器的数据可能已经失效，插值修补的意义不大，建议直接剔除该时段。
    - Cursor 指令：请帮我生成一份数据质量报告，包含各列的缺失值占比、均值和极值，并用绘图库（如 missingno）可视化缺失值的分布。
  - 2. 物理阈值过滤（Physical Range Filtering）
    - 这是暖通工程数据的“硬约束”。传感器坏了可能会传回 -999 或 999 这样的数值。
    - 操作：根据物理常识设定取值范围。
    - 逻辑示例：
    - 室内温度：10°C< T < 40°C
    - 冷冻水供水温度：5°C < T < 20°C
    - Cursor 指令：请根据物理常识过滤数据：如果温度超出 0 到 50 度范围，或相对湿度不在 0% 到 100% 之间，将其设为空值（NaN）。
  - 3. 时间尺度对齐与重采样（Resampling & Alignment）
    - 这是最容易被忽视的一步。冷机电流可能是每 1 分钟记一次，但室外温度可能每 1 小时才记一次。
    - 操作：将所有传感器的数据对齐到统一的时间刻度（如每 15 分钟一个点）。
    - 暖通关注点：使用 mean()（平均值）或 median()（中位数）进行重采样，平滑瞬时波动。
    - Cursor 指令：数据中的采样频率不统一，请以 15 分钟为基准进行重采样（resample），并用均值填充。
  - 4. 逻辑一致性检查（Physical Logic Check）
    - 利用暖通热力学原理检查数据之间的逻辑。

数据清晰我一共进行了七步，在开始之前可以设置你的 rules，也就是确保你每一步代码运行都是遵循一下的逻辑，不管你是不是提示它，只要它发现不符合逻辑的地方就会改掉，你需要设置整个必须的逻辑：比如活人总是热的什么的，等等见下图。

```
你现在是一名资深的暖通空调(HVAC)数据分析专家。在处理本项目时，请遵守以下规则：

1. 核心参数映射 (AHU 缩写):
   - Set Temperature = 设定温度目标值
   - Ventilation Temperature = 通风/混合风温度 (MA)
   - Supply Air Temperature = 送风温度 (SA)
   - Valve Position = 水阀开度 (0-100%)
   - Heating Supply Temperature = 热水供水温度

2. 物理常识约束 (逻辑校验):
   - 送风温度范围：冬季供暖模式通常在 30-50°C;夏季制冷模式通常在 12-18°C。若出现 <0°C 或 >70°C 视为传感器坏点。
   - 供回水逻辑：如果是供暖工况，加热盘管的供水温度 (Heating Supply Temperature) 必须大于送风温度 (SA)。
   - 联动逻辑：当送风机 (Supply Fan) 为 0(关闭)时，送风温度 (SA) 应该逐渐向环境温度靠拢，且流量相关数据应接近 0。

3. 数据清洗专业指导:
   - 缺失值处理：温度等连续信号优先使用 `df.interpolate(method='linear')`；风机/水泵开关等离散信号使用 `df.fillna(method='ffill')`。
   - 异常值识别：若温度在 1 分钟内跳变超过 5°C,通常视为电磁干扰噪声，需进行平滑处理。
   - 标签处理：关注 'Labeling' 列，'Sensor fault' 标记的时段数据不应参与模型训练。

4. 可视化规范:
   - 使用 matplotlib 或 seaborn,图表需包含清晰的中英文标注。
   - 在展示温度曲线时，务必将“设定温度”与“实际送风温度”放在同一张图对比。
```

- 第一步数据的读取：首先我要确报我的 cursor 中的 pandas 库可以调用并且，直接读取到我的 Excel 文件内容，于是就有了下图。

生成 ＋ 代码 ＋ Markdown | 全部运行 清除所有输出 | 大纲 ··· 选择内核

```python
import pandas as pd

# 1. 读取文件
file_path = 'combined_FDD (1).xls'
df = pd.read_excel(file_path)

# 2. 展示前 10 行
print("数据读取成功！")
df.head(10)
```

[3] Python

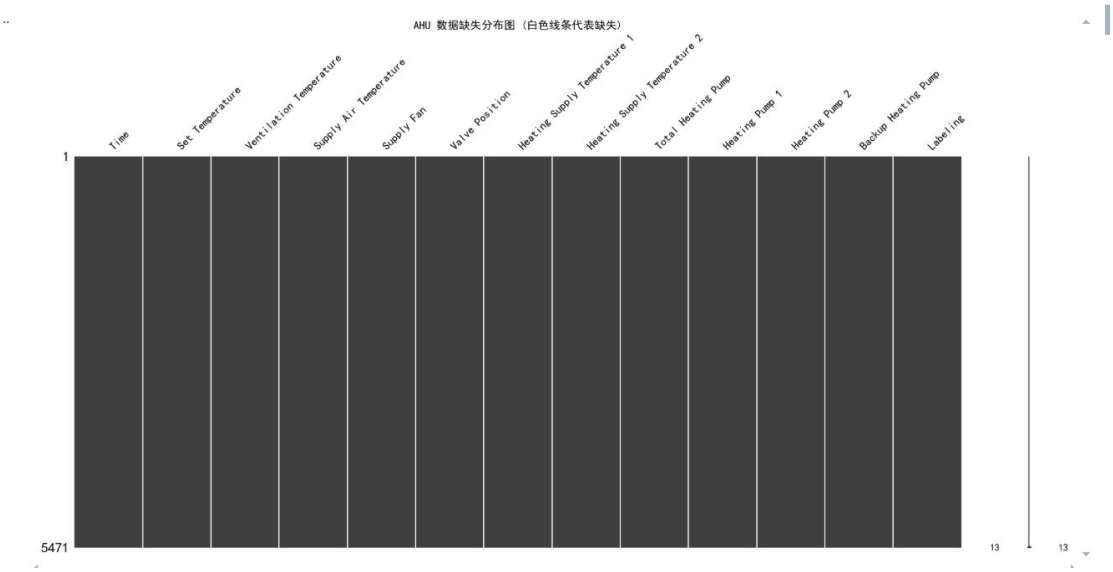
···

数据读取成功！

	Time	Set Temperature	Ventilation Temperature	Supply Air Temperature	Supply Fan	Valve Position	Heating Supply Temperature 1	Heating Supply Temperature 2	Total Heating Pump	Heating Pump 1	Heating Pump 2	Backup Heating Pump	Labeling
0	2024-01-01 00	18	22.57	35.70	1.0	10.0	42.50	42.27	2.0	1.0	0.0	1.0	Sensor fault
1	2024-01-01 01	18	22.36	35.63	1.0	10.0	42.41	42.33	2.0	1.0	0.0	1.0	Sensor fault
2	2024-01-01 02	18	22.22	35.52	1.0	10.0	42.87	42.24	2.0	1.0	0.0	1.0	Sensor fault
3	2024-01-01 03	18	22.12	35.55	1.0	10.0	42.99	42.21	2.0	1.0	0.0	1.0	Sensor fault
4	2024-01-01 04	18	22.04	35.47	1.0	10.0	42.70	42.25	2.0	1.0	0.0	1.0	Sensor fault
5	2024-01-01 05	18	22.09	35.54	1.0	10.0	42.43	42.32	2.0	1.0	0.0	1.0	Sensor fault
6	2024-01-01 06	18	22.08	35.59	1.0	10.0	42.40	42.27	2.0	1.0	0.0	1.0	Sensor fault
7	2024-01-01 07	18	22.13	35.53	1.0	10.0	43.04	42.27	2.0	1.0	0.0	1.0	Sensor fault
8	2024-01-01 08	18	22.11	35.59	1.0	10.0	42.99	42.28	2.0	1.0	0.0	1.0	Sensor fault

- 第二步是统计质量报告：利用 python 中的 pandas 将初始数据文件进行全局扫描，之后求出相关参数的最大值、最小值、平均值、缺失值以及缺失率。

	缺失值数量	缺失率 (%)	平均值	最小值	最大值
Time	0	0.0	NaN	NaN	NaN
Set Temperature	0	0.0	18.86	18.00	30.00
Ventilation Temperature	0	0.0	13.34	-1.63	28.13
Supply Air Temperature	0	0.0	24.51	8.58	47.92
Supply Fan	0	0.0	0.20	0.00	1.00
Valve Position	0	0.0	14.42	8.46	100.00
Heating Supply Temperature 1	0	0.0	37.67	19.57	58.84
Heating Supply Temperature 2	0	0.0	36.35	18.95	54.38
Total Heating Pump	0	0.0	1.24	0.25	2.00
Heating Pump 1	0	0.0	0.11	0.00	1.00
Heating Pump 2	0	0.0	0.35	0.00	1.00
Backup Heating Pump	0	0.0	0.79	0.00	1.00
Labeling	0	0.0	NaN	NaN	NaN



- 第三步是数值过滤：需要检验温度列的温度是否正常，或者检验控制温度的阀门是否正常。设置合理的范围对不符合的数据进行剔除。

```python
# 1. 定义需要校验的温度列
temp_columns = [
    'Set Temperature',
    'Ventilation Temperature',
    'Supply Air Temperature',
    'Heating Supply Temperature 1',
    'Heating Supply Temperature 2'
]

# 2. 执行过滤：超出 0~50℃ 的设为 NaN
for col in temp_columns:
    # 统计过滤前的异常数
    invalid_count = df[(df[col] < 0) | (df[col] > 50)].shape[0]

    if invalid_count > 0:
        print(f"⚠️ 列 [{col}] 发现 {invalid_count} 个超出物理范围 (0-50℃) 的异常值，已设为空值。")
        # 将异常值替换为 NaN
        df.loc[(df[col] < 0) | (df[col] > 50), col] = np.nan
    else:
        print(f"✅ 列 [{col}] 数据均在 0-50℃ 合理范围内。")

# 3. 校验阀门开度：必须在 0~100% 之间
valve_col = 'Valve Position'
invalid_valve = df[(df[valve_col] < 0) | (df[valve_col] > 100)].shape[0]
if invalid_valve > 0:
    df.loc[(df[valve_col] < 0) | (df[valve_col] > 100), valve_col] = np.nan
    print(f"⚠️ 列 [{valve_col}] 发现 {invalid_valve} 个非法开度值，已修正。")

print("\n✨ 第二步：物理过滤完成！")
```

Python

```
✅ 列 [Set Temperature] 数据均在 0-50℃ 合理范围内。
✅ 列 [Ventilation Temperature] 数据均在 0-50℃ 合理范围内。
✅ 列 [Supply Air Temperature] 数据均在 0-50℃ 合理范围内。
⚠️ 列 [Heating Supply Temperature 1] 发现 62 个超出物理范围 (0-50℃) 的异常值，已设为空值。
⚠️ 列 [Heating Supply Temperature 2] 发现 7 个超出物理范围 (0-50℃) 的异常值，已设为空值。
```

- 第四步是时间尺度的对齐和重采样：为了保证数据都可以统一有效的处理，我们需要先定义一个统一的时间尺度，用它作为衡量工况运行的具体的时间，这个格式必须先统一，要不然代码很难读懂它。之后我进行了重采样，因为数据是关于韩国一个月内的空调运行的工况，且每小时记录一次，这个数据还是太少不能够支撑我得到全部想要的结果，所以我将一小时分为了四段，每十五分钟设置一次数据，通过线性拟合，可以自己填充数据内容。

不过在线性拟合的过程中很有可能会覆盖掉原来数据的故障点，所以我在其中加了判断条件，如果 15 分钟内热力学的曲线在极短的时间内发生了跳动，我

就认为这个点是故障点，故障点就会保留下来。

```
📊 正在扫描 5471 行数据中的'微观特征'...
- 发现非正常状态样本：467 行
✅ 智能对齐完成！
- 保留了 15 分钟窗口内的'最严重'信号，避免了故障隐身。
```

将像现在图片中显示的一样，运行结果显示“非正常状态的样本是 467 行——这个是在供暖的工况下（也就是通过供回水的温度来进行判定的）”，当然我也运行过制冷的工况下，我记得当时显示异常数据有 2000 多条，所以代码逻辑必然是有欠缺的，现在的代码逻辑只会单纯的识别到供暖或者制冷，所以我又改进了代码，设置为可以同时识别制冷或者是供暖的情况然后进行分类。

```
📊 逻辑一致性终极检查：
- 运行模式分布：
运行模式
停机/平衡 (Standby)    1639
供暖模式 (Heating)     1044
制冷模式 (Cooling)      274
Name: count, dtype: int64
- 物理逻辑异常行数：20 (0.68%)
```

- 第五步是突变点的识别：利用 3-Sigma 准则（偏离均值不能超过 3 倍标准差的点）识别数据中的突变点，检查温度列是否有突变。如果有的话意味着你的室温很可能从 5℃在极短的时间内突破到 20℃这很显然也是不对的。

```
📊 突变点扫描报告 (3-Sigma 准则)：
- 列 [Set Temperature]: 发现 0 个突变点
- 列 [Ventilation Temperature]: 发现 0 个突变点
- 列 [Supply Air Temperature]: 发现 0 个突变点
- 列 [Heating Supply Temperature 1]: 发现 0 个突变点
- 列 [Heating Supply Temperature 2]: 发现 0 个突变点
```

但是由于我在前面的过程中进行了平滑的十五分钟采样，很可能导致了突变点被覆盖，所以我又对刚开始的初始参数进行了扫描检查。

```
🔍 微观扫描报告：
- 在原始 5471 行数据中，共发现 0 个瞬时突变点。
✅ 深度确认：原始数据中确实没有显著的瞬时跳变，说明你的传感器信号非常稳定。

✨ 已将【波动强度】作为新特征补充进主表，现在你可以通过'波动强度'看到被抹平的真相了！
```

之后在代码中插入了机器学习的开源学习库

```
!pip install scikit-learn
```

```
Collecting scikit-learn
  Downloading scikit_learn-1.8.0-cp311-cp311-win_amd64.whl.metadata (11 kB)
Requirement already satisfied: numpy>=1.24.1 in c:\users\史浩东\appdata\local\programs\python\python311\lib\site-packages (from scikit-learn) (2.2.6)
Requirement already satisfied: scipy>=1.10.0 in c:\users\史浩东\appdata\local\programs\python\python311\lib\site-packages (from scikit-learn) (1.16.3)
Collecting joblib>=1.3.0 (from scikit-learn)
  Downloading joblib-1.5.3-py3-none-any.whl.metadata (5.5 kB)
Collecting threadpoolctl>=3.2.0 (from scikit-learn)
  Downloading threadpoolctl-3.6.0-py3-none-any.whl.metadata (13 kB)
Downloading scikit_learn-1.8.0-cp311-cp311-win_amd64.whl (8.1 MB)
   ---------------------------------------- 0.0/8.1 MB ? eta -:--:--
   ---------------------------------------- 0.0/8.1 MB ? eta -:--:--
   ---------------------------------------- 0.0/8.1 MB ? eta -:--:--
   - -------------------------------------- 0.3/8.1 MB ? eta -:--:--
   --- ------------------------------------ 0.8/8.1 MB 1.9 MB/s eta 0:00:04
   --- ------------------------------------ 0.8/8.1 MB 1.9 MB/s eta 0:00:04
   ------------------- -------------------- 3.9/8.1 MB 5.2 MB/s eta 0:00:01
   ---------------------------------------- 8.1/8.1 MB 8.6 MB/s  0:00:01
Downloading joblib-1.5.3-py3-none-any.whl (309 kB)
Downloading threadpoolctl-3.6.0-py3-none-any.whl (18 kB)
Installing collected packages: threadpoolctl, joblib, scikit-learn

   ------------- -------------------------- 1/3 [joblib]
   ------------- -------------------------- 1/3 [joblib]
   ------------------------- ------------- 2/3 [scikit-learn]
   ------------------------- ------------- 2/3 [scikit-learn]
...
   ------------------------- ------------- 2/3 [scikit-learn]
   ---------------------------------------- 3/3 [scikit-learn]

Successfully installed joblib-1.5.3 scikit-learn-1.8.0 threadpoolctl-3.6.0
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...

[notice] A new release of pip is available: 25.3 -> 26.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

- 第六步是对于缺失值的智能插补，这一步会先需要检查目前的文档内容中是否有缺失值，之后利用上面导入的机器学习的开源库进行拟合回归，把有缺失的部分插入进去

```
✅ 检查完毕：当前目标列在参考列完备的情况下没有缺失值。
C:\Users\史浩东\AppData\Local\Temp\ipykernel_8788\686473851.py:37: FutureWarning:
  df = df.interpolate(method='linear', limit=2)
```

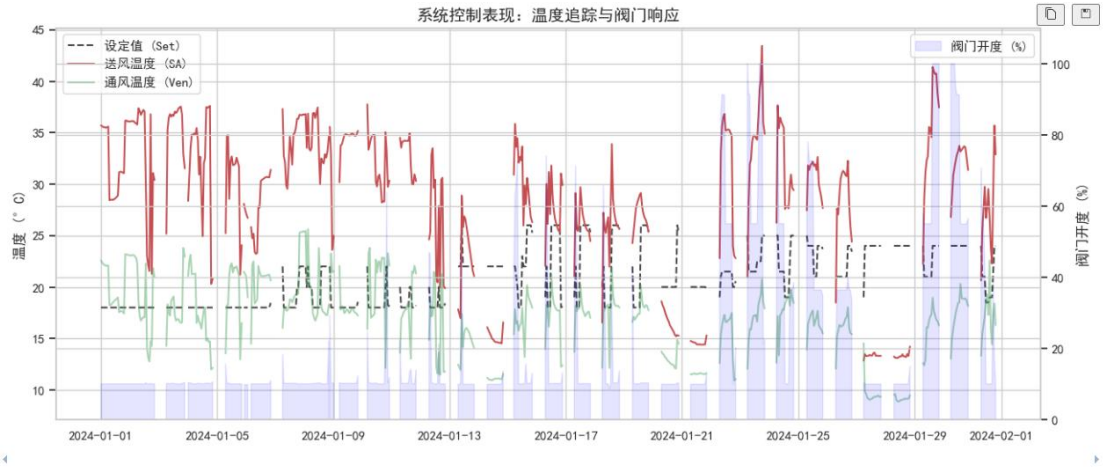
- 第七步就是特征值的归回，包括了提取时间参数、计算供回水的温差来考虑换热量的大小、计算送风偏差反应控制系统的运行、通过供水侧来计算负荷指标等等。

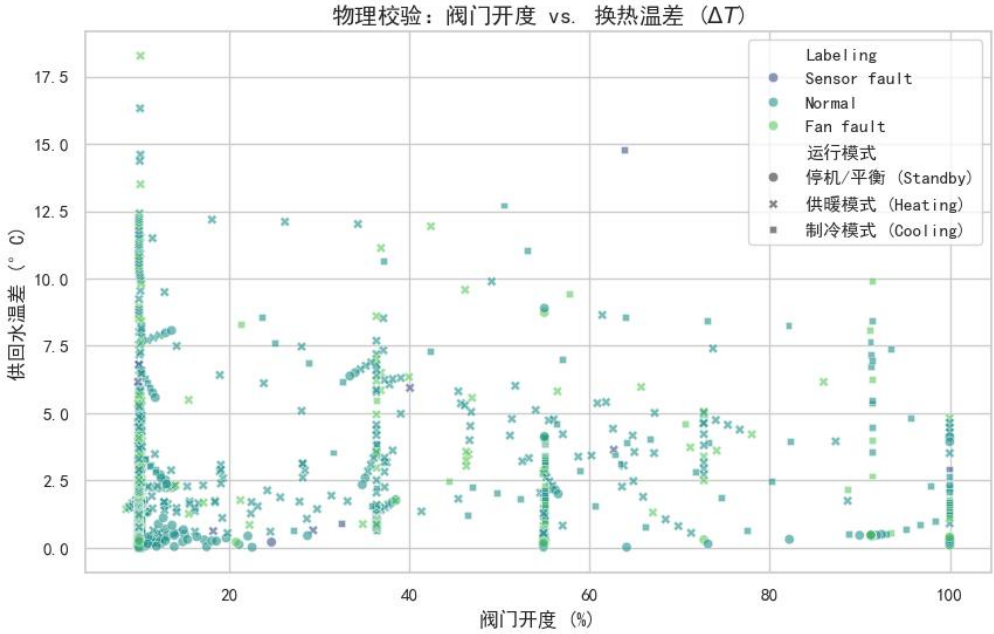
```
✅ 第七步：特征衍生完成！
- 新增时间特征：小时、星期几、是否为工作日
- 新增物理特征：供回水温差 (Delta_T)、送风偏差、负荷指标
```

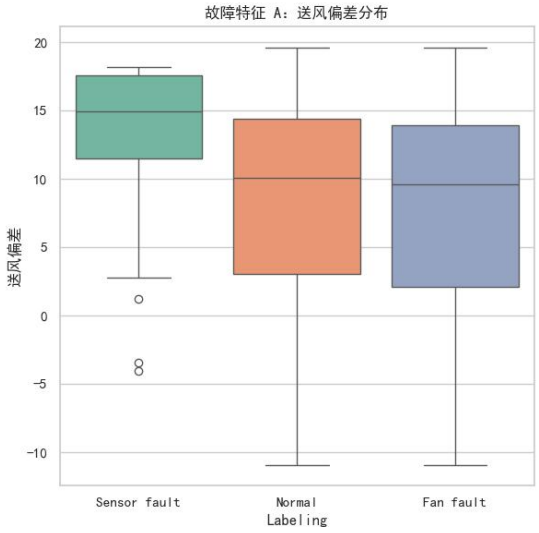
Time	小时(Hour)	是否为工作日(IsWorkday)	供回水温差(Delta_T)	送风偏差	Labeling
2024-01-01 00:00:00	0	True	0.2300	17.7000	Sensor fault
2024-01-01 00:15:00	0	True	0.1925	17.6825	Normal
2024-01-01 00:30:00	0	True	0.1550	17.6650	Normal
2024-01-01 00:45:00	0	True	0.1175	17.6475	Normal
2024-01-01 01:00:00	1	True	0.0800	17.6300	Sensor fault

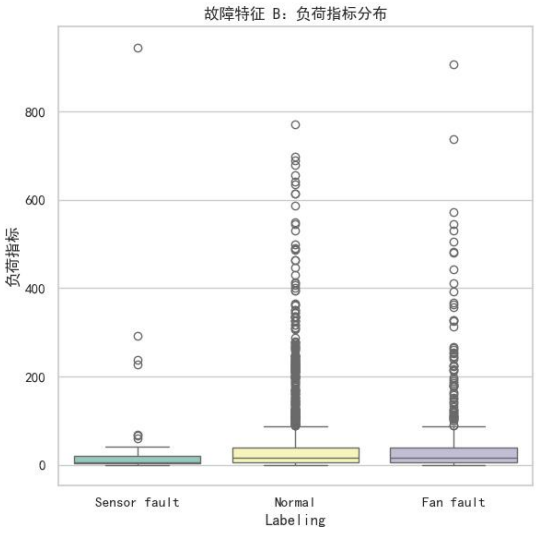
5. 数据输出的结果

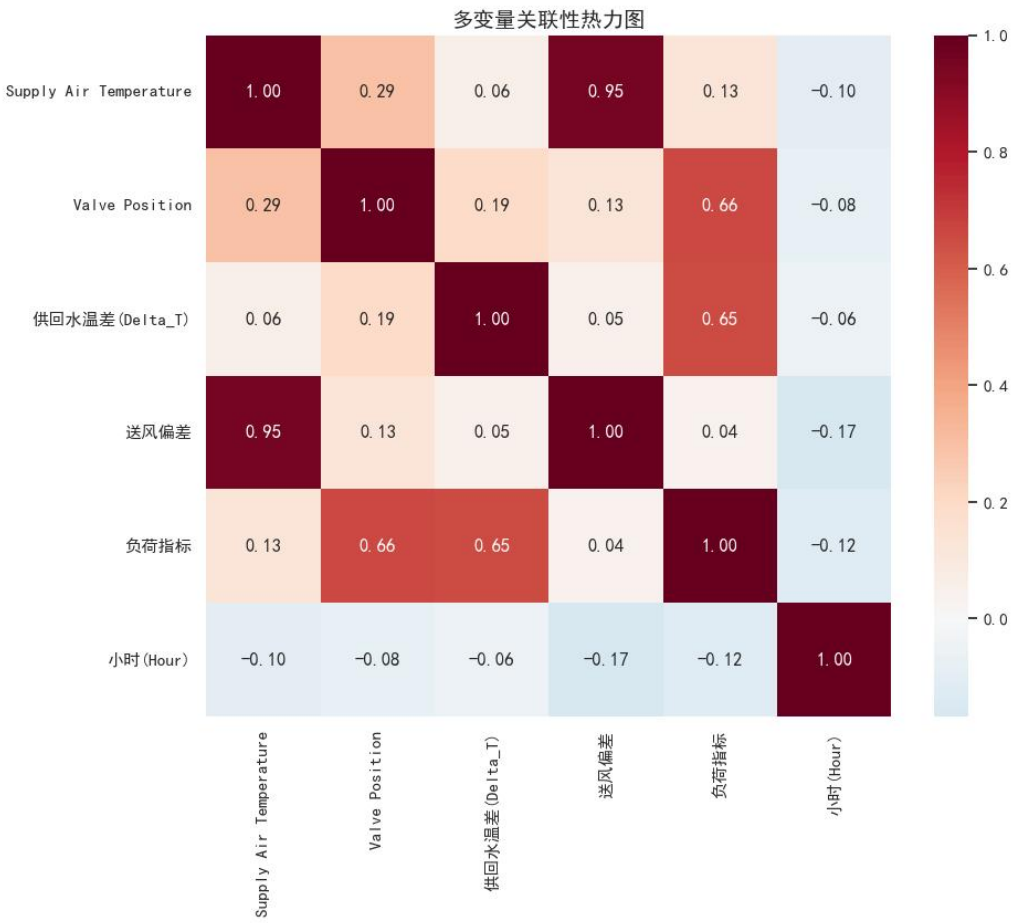
结果的输出包括了数据的输出，以及绘制的图表输出











6. 异常分析

对于异常点可以提取出来，深度分析找到数据中反应出的关联故障。可以结合时间，对比该时间下阀门的控制器反应，最后对比环境温度参数得到结论。

📊 切片分析报告：
- 1月29日连续饱和故障点数：7（阀门 99.98% 持续时间约 1.5小时）
- 全表'指令饱和但无换热'点数：79

📍 1月29日故障段细节（典型执行器卡死特征）：

Time	Valve Position	供回水温差(Delta_T)	Supply Air Temperature
2024-01-29 15:45:00	99.98	0.1825	41.0725
2024-01-29 16:00:00	99.98	0.1200	40.9700
2024-01-29 16:15:00	99.98	0.1150	40.9025
2024-01-29 16:30:00	99.98	0.1100	40.8350
2024-01-29 16:45:00	99.98	0.1050	40.7675
2024-01-29 17:00:00	99.98	0.1000	40.7000
2024-01-29 17:15:00	99.98	0.1750	40.7125

分析报告：

AHU_Analysis_Report.md - 记事本

文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

```
# AHU 传感器数据预处理与初步诊断报告
**报告生成时间**：2026-04-05
**研究对象**：韩国 AHU 实验数据集（2024年1月）

---

## 1. 数据预处理概况
- **样本规模**：从原始 5471 行重采样并对齐至 2957 行（15分钟步长）。
- **质量检查**：
  - 物理量程过滤：已剔除超出 0-50°C 的异常值。
  - 智能插补：采用基于 **Scikit-learn** 的线性回归模型（SA = f(T1, Valve)）完成缺失值填补。
  - 突变扫描：经 3-Sigma 滑动窗口检测，原始数据无显著高频瞬时尖峰，信号质量平稳。

## 2. 特征工程结果
- **时间维度**：已提取小时、星期几、工作日标签。
- **物理指标**：
  - 计算了 **供回水温差 ($\Delta T_{water}$)** 以衡量换热强度。
  - 计算了 **送风偏差 (Supply Air Deviation)** 以量化控制精度。
  - 计算了 **负荷指标 (Load Index)** 用于多变量关联分析。

## 3. 逻辑一致性与故障诊断 (FDD)
- **工况识别**：
  - 供暖模式 (Heating)：1044 行
  - 制冷模式 (Cooling)：274 行
  - 停机/平衡 (Standby)：1639 行
- **逻辑异常发现**：共识别出 **20 个** 显著物理逻辑冲突点。
- **核心故障特征**：
  - **系统静差**：送风温度长期偏离设定值，平均偏差达 **8.09°C**。
  - **指令-反馈脱节**：在 1 月 29 日等时段，阀门开度饱和（99.98%）但温差几乎为零（< 0.2°C），判定为执行器卡死或热源丢失故障。

## 4. 结论与建议
1. **数据可用性**：清洗后的数据集逻辑自洽，特征维度丰富，完全满足机器学习建模（如随机森林、LSTM）的需求。
2. **重点研究方向**：建议后续针对“高送风偏差”背后的传感器偏置（Bias Fault）进行定量补偿研究。
```

第 1 行，第 1 列 100% Windows (CRLF) UTF-8